

tmx Mouse-Copy + Double-Prompt + B3 Closure + Dual-Host Release Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (– []) syntax for tracking.

Goal: Fix the two user-reported defects (mouse select/copy unusable in tmx panes esp. under Claude Code; double session-name prompt on new bash-login terminals), close the stale B3 anti-bluff entry with current evidence, cover every change with fully-automatic physical-proof tests, re-run `setup.sh` on Mistborn (macOS) + nezha (Linux) deploying the latest tmx, validate/verify on both, document, and release a new version via GitHub + GitLab CLIs with a comprehensive changelog.

Architecture: Both bugs are fixed at the SOURCE template layer (§11.4.1): `scripts/tmx-shell-init.sh.template` (double-prompt) and `scripts/tmux.conf.template` (mouse). New runtime tests live in `scripts/tests/`. Validation is fully autonomous (§11.4.98) via PTY harnesses and headless tmux servers driven through the same code path operators use. Release follows VERSION single-source-of-truth + `commit_all.sh`.

Tech Stack: POSIX sh / bash, tmux 3.6a config, Python3 pty (test harness, stdlib only), osascript (iTerm2 repro/proof), cliclick (optional real-mouse proof; SKIP-with-reason if absent), gh + glab CLIs (release).

Confirmed root causes (Phase 1, with reproduced physical proof):

- **Bug #2 (double prompt):** On Linux/bash a single login-shell PROCESS sources `tmx-shell-init.sh` twice — `.bash_profile` carries the source line AND sources `.bashrc` which also carries it. The blank/default path returns (does not exec), so `.bash_profile` continues and the second source re-prompts. Reproduced: nezha bash -l -i → PROMPT_COUNT=2; mac zsh → 1.
 - **Bug #1 (mouse copy):** Config is correct & live (not stale): live Herald server has mouse on, root M-/S-MouseDrag1Pane → copy-mode -M, @clip→pbcopy verified working, in-tmux keyboard copy verified. In mouse-tracking apps (Claude Code) root MouseDrag1Pane forwards plain drag to the app; Alt-drag is consumed by iTerm2's native-selection bypass (Option Key Sends=0), leaving only Shift-drag reaching tmux — undiscoverable to the user.
 - **B3:** Likely already addressed in-tree (tests 49/50 added, meta-test M20/M21 retargeted to the Darwin rlimit wrapper, M22 has an §11.4.3 SKIP). Issues.md entry is stale. CONFIRM via a fresh meta-test run before closing (§11.4.7).
-

Task 1: Bug #2 — per-process idempotency guard in tmx-shell-init

Files:

- Modify: scripts/tmx-shell-init.sh.template (insert guard immediately before the prompt printf)
- Test: scripts/tests/51_double_prompt_idempotent.sh (new)
- Regen: scripts/tmx-shell-init.sh (via setup.sh regeneration, Task 7)



Step 1: Write the failing test — scripts/tests/51_double_prompt_idempotent.sh

A self-driving PTY harness that simulates a bash login shell sourcing the init TWICE in one process (the exact nezha topology) and asserts the prompt appears exactly once. Uses a sandbox HOME so it never touches the operator's dotfiles (§11.4.14).

```
#!/bin/sh
# 51_double_prompt_idempotent.sh — §11.4.1/§11.4.98 regression for
# the
# bash-login double session-name prompt (user report 2026-05-29;
# nezha
# bash -l -i reproduced PROMPT_COUNT=2). Fully autonomous, re-
# runnable.

set -eu
SELF_DIR=$(CDPATH= cd -- "$(dirname -- "$0")" && pwd)
REPO_ROOT=$(CDPATH= cd -- "$SELF_DIR/../../.." && pwd)
INIT="$REPO_ROOT/scripts/tmx-shell-init.sh"
[ -r "$INIT" ] || { echo "FAIL: $INIT missing (run setup.sh)";
    exit 1; }

SANDBOX=$(mktemp -d)
trap 'rm -rf "$SANDBOX"' EXIT
# .bash_profile carries the init AND sources .bashrc which also
# carries it —
# the exact nezha double-source topology, in one process.
cat > "$SANDBOX/.bashrc" <<RC
[ -r "$INIT" ] && . "$INIT"
RC
cat > "$SANDBOX/.bash_profile" <<RC
[ -r "$INIT" ] && . "$INIT"
if [ -f "\$HOME/.bashrc" ]; then . "\$HOME/.bashrc"; fi
RC

python3 - "$SANDBOX" "$INIT" <<'PY'
import os, pty, select, time, sys
sandbox = sys.argv[1]
PROMPT = b"Enter session name"
env = dict(os.environ); env["HOME"] = sandbox; env.pop("TMUX",
    None)
# tmx must be reachable for the prompt path to run.
```

```

env["PATH"] = os.path.dirname(os.path.dirname(sys.argv[2])) + ":"
                + env.get("PATH","")
pid, fd = pty.fork()
if pid == 0:
    os.execvpe("/bin/bash", ["/bin/bash","-l","-i"], env);
    os._exit(127)
buf=b""; sent=False; last=time.time(); sent_exit=False
while True:
    r,_,_=select.select([fd],[],[],0.3)
    if fd in r:
        try: d=os.read(fd,4096)
        except OSError: break
        if not d: break
        buf+=d; last=time.time()
        if (not sent) and PROMPT in buf:
            time.sleep(0.4); os.write(fd,b"\n"); sent=True
    elif time.time()-last>5:
        if not sent_exit:
            try: os.write(fd,b"exit\n")
            except OSError: pass
            sent_exit=True; last=time.time()
        else: break
try: os.close(fd)
except OSError: pass
try: os.waitpid(pid,0)
except OSError: pass
n = buf.count(PROMPT)
print("EVIDENCE prompt_count=%d" % n)
sys.exit(0 if n==1 else 2)
PY
rc=$?
[ "$rc" -eq 0 ] && echo "PASS: 51 double-prompt idempotent
(exactly 1 prompt per process)" \
    || echo
"FAIL: 51 double-prompt - expected 1 prompt, see EVIDENCE
above"
exit $rc

```



Step 2: Run test to verify it FAILS (before the source fix, with current generated script that lacks the guard)

Run: `bash scripts/tests/51_double_prompt_idempotent.sh` Expected: EVIDENCE prompt_count=2 then FAIL (exit 2) — proves the test catches the real defect.



Step 3: Apply the source fix in `scripts/tmx-shell-init.sh.template`, inserting immediately before the `# Prompt + read.` line:

```

# Per-process idempotency guard. A single shell PROCESS can source
  this
# script more than once (bash login shells where .bash_profile
  sources

```

```

# tmx-shell-init AND sources .bashrc which sources it again -
# forensic
# anchor: nezha `bash -l -i` double-prompt, user report
# 2026-05-29,
# reproduced PROMPT_COUNT=2). The blank/`default` path RETURNS
# (does not
# exec), so .bash_profile then continues and the second source re-
# prompts.
# A NON-exported marker persists across sources WITHIN this
# process but
# resets for each NEW shell process, so every new terminal still
# prompts
# exactly once. (§11.4.1 fix at source, not at the rc call sites.)
if [ -n "${_TMX_SHELL_INIT_PROMPTED:-}" ]; then
    return 0 2>/dev/null || exit 0
fi
_TMX_SHELL_INIT_PROMPTED=1

```



Step 4: Regenerate the live script so the test (which sources the generated file) sees the fix:

Run: `bash scripts/setup.sh --verify-only` is NOT enough; regenerate the shell-init specifically. If `setup.sh` has no targeted flag, run the full `bash scripts/setup.sh` (Task 7 does this on both hosts). For local TDD, mirror the edit into `scripts/tmx-shell-init.sh` between the same anchor lines. Expected: generated `scripts/tmx-shell-init.sh` contains `_TMX_SHELL_INIT_PROMPTED`.



Step 5: Run test to verify it PASSES

Run: `bash scripts/tests/51_double_prompt_idempotent.sh` Expected: EVIDENCE `prompt_count=1` then PASS (exit 0).



Step 6: Re-runability proof (§11.4.50/§11.4.98)

Run: `for i in 1 2 3; do bash scripts/tests/51_double_prompt_idempotent.sh || exit 1; done` Expected: PASS all 3.



Step 7: Companion doc + commit — update docs/scripts/ companion for the shell-init if present (§11.4.18); commit via `bash commit_all.sh "..."` (do NOT git push directly).

Task 2: Bug #2 — paired §1.1 meta-test mutation

Files:

- Modify: `scripts/tests/meta_test_false_positive_proof.sh` (add mutation M-DBLPROMPT)



Step 1: Add the mutation — strip the guard, assert test 51 FAILs, restore, assert PASS:

```
# — M-DBLPROMPT: strip the per-process idempotency guard from
    shell-init —
MDP_TARGET="$REPO_ROOT/scripts/tmx-shell-init.sh"
if [ ! -f "$MDP_TARGET" ]; then
    _skip "M-DBLPROMPT: $MDP_TARGET not present (run setup.sh)"
else
    _run_mutation \
        "M-DBLPROMPT: strip per-process idempotency guard from
        tmx-shell-init" \
        "$MDP_TARGET" \
        "inplace_sed '/_TMX_SHELL_INIT_PROMPTED=1/d; /if \[ -n \"\$
        ${_TMX_SHELL_INIT_PROMPTED:-}\" \]/,/^fi/d' \"\$
        $target_abs\" \" \" \
        "bash \"$REPO_ROOT/scripts/tests/
        51_double_prompt_idempotent.sh\" \" \" \
        "FAIL"
fi
```

(Adapt the helper-call shape to the harness's existing `_run_mutation / inplace_sed` conventions — read the file's existing mutations first.)



Step 2: Run meta-test, confirm M-DBLPROMPT is CAUGHT
(mutation→FAIL, revert→PASS).

Run: `bash scripts/tests/meta_test_false_positive_proof.sh 2>&1 | grep -i DBLPROMPT Expected: MUTATION CAUGHT + FEATURE RESTORED/INTACT.`



Step 3: Commit via `commit_all.sh`.

Task 3: Bug #1 — mouse-toggle escape hatch + discoverability in `tmux.conf`

Files:

- Modify: `scripts/tmux.conf.template` (add prefix + m toggle + clarifying comments + optional status hint)
- Test: `scripts/tests/52_mouse_toggle_and_copy.sh` (new)



Step 1: Write the failing test — `scripts/tests/52_mouse_toggle_and_copy.sh`

Headless, autonomous. Boots a tmux server with the template config (built binary if present, else system tmux), asserts: (a) prefix+m toggle binding exists and actually flips mouse; (b) the in-tmux copy pipe reaches the clipboard backend via the real @clip path; (c) the tracking-app modifier overrides resolve.

```
#!/bin/sh
# 52_mouse_toggle_and_copy.sh – $11.4.98 autonomous proof that
# mouse
# select/copy is usable: prefix+m toggles mouse for native
# selection, and
# the @clip copy pipe delivers to the clipboard backend. Re-
# runnable.

set -eu
SELF_DIR=$(CDPATH= cd -- "$(dirname -- "$0")" && pwd)
REPO_ROOT=$(CDPATH= cd -- "$SELF_DIR/../../.." && pwd)
CONF="$REPO_ROOT/scripts/tmux.conf.template"
BIN="$REPO_ROOT/tmux/build-darwin/bin/tmux"; [ -x "$BIN" ] ||
    BIN="$REPO_ROOT/tmux/build-linux/bin/tmux"; [ -x "$BIN" ] ||
    BIN=$(command -v tmux)
[ -n "$BIN" ] || { echo "FAIL: no tmux binary"; exit 1; }
L=mouse52probe
"$BIN" -L "$L" kill-server 2>/dev/null || true
"$BIN" -L "$L" -f "$CONF" new-session -d -s p -x 80 -y 24
trap "'$BIN' -L '$L' kill-server 2>/dev/null || true" EXIT
sleep 0.4
fail=0
# (a) toggle binding present
"$BIN" -L "$L" list-keys -T prefix 2>/dev/null | grep -qE
    '[:space:]]m[:space:]].*set.*mouse' \
    && echo "EVIDENCE: prefix+m toggle binding present" || { echo
        "FAIL: no prefix+m mouse toggle"; fail=1; }
# (a2) toggle actually flips mouse
"$BIN" -L "$L" set -g mouse on
"$BIN" -L "$L" send-keys -X cancel 2>/dev/null || true
before=$("$BIN" -L "$L" show -g mouse)
"$BIN" -L "$L" set -g mouse off
after=$("$BIN" -L "$L" show -g mouse)
[ "$before" != "$after" ] && echo "EVIDENCE: mouse flips ($before
    -> $after)" || { echo "FAIL: mouse did not flip";
    fail=1; }
"$BIN" -L "$L" set -g mouse on
# (c) modifier overrides resolve
"$BIN" -L "$L" list-keys 2>/dev/null | grep -qE -- '-T root +S-
    MouseDrag1Pane' \
    && echo "EVIDENCE: Shift-drag override present" || { echo
        "FAIL: no Shift-drag override"; fail=1; }
# (b) copy pipe reaches a clipboard backend (use a temp sink to
# stay autonomous/headless)
SINK=$(mktemp); CLIP="cat > $SINK"
"$BIN" -L "$L" set -g @cliptest "$CLIP"
"$BIN" -L "$L" send-keys -l 'echo MOUSE52_COPY_PROOF'; "$BIN" -L
    "$L" send-keys Enter; sleep 0.3
"$BIN" -L "$L" copy-mode
"$BIN" -L "$L" send-keys -X history-top
"$BIN" -L "$L" send-keys -X begin-selection
```

```

"$BIN" -L "$L" send-keys -X bottom-line
"$BIN" -L "$L" send-keys -X end-of-line
"$BIN" -L "$L" send-keys -X copy-pipe-and-cancel "#{@cliptest}"
sleep 0.4
grep -q 'MOUSE52_COPY_PROOF' "$SINK" && echo "EVIDENCE: copy pipe
delivered ($(tr -d '\n' < $SINK | head -c 60))" || { echo
"FAIL: copy pipe empty"; fail=1; }
rm -f "$SINK"
[ "$fail" -eq 0 ] && echo "PASS: 52 mouse toggle + copy usable"
|| echo "FAIL: 52 mouse toggle/copy"
exit $fail

```



Step 2: Run to verify it FAILS (before the conf fix — no prefix+m binding yet)

Run: `bash scripts/tests/52_mouse_toggle_and_copy.sh` Expected: FAIL: no prefix+m mouse toggle.



Step 3: Apply conf fix in `scripts/tmux.conf.template`, near the mouse section:

— Quick mouse toggle: prefix+m

```

# Robust, terminal-agnostic copy escape hatch. With mouse OFF, the
outer
# terminal's NATIVE selection (iTerm2 / Terminal.app drag → Cmd-
C / right-
# click copy) works EVERYWHERE — including inside mouse-tracking
apps like
# Claude Code, where tmux otherwise forwards the drag to the app.
Toggle
# back on to restore tmux scrollback + copy-mode. (forensic
anchor: user
# report 2026-05-29 "cannot select/copy with mouse, especially in
claude".)
bind m set -g mouse \; display-message 'mouse #{?mouse,ON (tmux
selection),OFF (native terminal selection - drag + Cmd-C)}'

```

Also expand the existing comment block to state plainly: *in mouse-tracking apps, hold **Shift** and drag to select within tmux (terminal-agnostic); or press prefix m to turn mouse off and use the terminal's own selection.*



Step 4: Run to verify it PASSES

Run: `bash scripts/tests/52_mouse_toggle_and_copy.sh` Expected: all EVIDENCE: lines + PASS.



Step 5: Re-runability (§11.4.50) — `for i in 1 2 3; do bash scripts/ tests/52_mouse_toggle_and_copy.sh || exit 1; done.`



Step 6: Commit via `commit_all.sh`.

Task 4: Bug #1 — real end-to-end mouse-drag physical proof (best-effort, §11.4.3 honest SKIP)

Files:

- Test: `scripts/tests/53_real_mouse_drag_copy.sh` (new, macOS-gated)



Step 1: Write a macOS-only test that, IF `cliclick` is available AND synthetic events are permitted, opens an iTerm2 window via `osascript`, starts a `tmx` session, writes known text, performs a real Shift-drag with `cliclick`, and asserts `pbpaste` contains the text. If `cliclick` is absent or events are blocked (accessibility), emit `SKIP: 53 real-mouse - cliclick/accessibility unavailable (§11.4.3 topology)` and exit 0-as-skip per the suite's SKIP convention. Never PASS-by-default; never FAIL-for-environment.



Step 2: Run; record whichever of PASS/SKIP applies with EVIDENCE (clipboard content or the precise unavailability reason). Document the result in the QA evidence dir `docs/qa/<run-id>/` (§11.4.83).



Step 3: Commit via `commit_all.sh`.

Task 5: B3 — confirm current meta-test state and close/refresh Issues.md

Files:

- Read: `qa-results/baseline_metatest_*.log` (the run launched at planning time)
- Modify: `Issues.md`, `Fixed.md` (+ summaries via `scripts/testing/sync_issues_docs.sh` or project equivalent)



Step 1: Read the completed baseline meta-test log. Record the exact `MUTATIONS CAUGHT / ESCAPED / SKIPPED` line and the per-line status of M20, M21, M22 (§11.4.7 same-conditions evidence).



Step 2 (decision):

- If M20/M21 no longer escape (retargeted + covered by tests 49/50) and M22 SKIPs-with-reason → migrate B3's M20/M21 portion to `Fixed.md`

with the meta-test evidence; reclassify the M22 portion to an explicit §11.4.3 SKIP note (closed) OR keep only the nezha-codegraph-baseline sub-item OPEN until Task 9 repairs it.

- If any genuinely still escapes → STOP (§11.4.4 interrupt), open the precise sub-defect, fix the test-design per Issues.md B3 closure conditions (tighten test 21/49 to isolate the script guard; re-architect test 18/50 to drive only auto-install), add paired mutations, re-run.



Step 3: Regenerate Issues/Fixed summaries + HTML/PDF via the project sync wrapper (§11.4.12/§11.4.53/§11.4.65). Commit via `commit_all.sh`.

Task 6: Full pre-deploy suite on Mistborn (macOS) — baseline GREEN



Step 1 (§11.4.89 background):

```
nohup bash scripts/tests/run_all.sh > qa-results/  
mistborn_runall_$(date +%Y%m%d_%H%M%S).log 2>&1 & disown
```



Step 2: Poll the log; on completion assert zero FAIL, every SKIP carries a reason. Include new tests 51/52/53.



Step 3: `nohup bash scripts/tests/
meta_test_false_positive_proof.sh > qa-results/mistborn_meta_$(
date +%Y%m%d_%H%M%S).log 2>&1 & disown`; assert zero ESCAPED (only reasoned SKIPS).



Step 4: If anything fails → §11.4.4 interrupt, fix at root, repeat from the top.

Task 7: Deploy latest tmx on Mistborn via `setup.sh` + post-deploy validation



Step 1: `bash scripts/install_deps.sh` (idempotent) then `bash scripts/
setup.sh` — full build + verify gate + regenerate `scripts/tmx`, `scripts/
tmx-shell-init.sh`, `~/.tmux.conf`.



Step 2: Assert the regenerated `scripts/tmx-shell-init.sh` contains `_TMX_SHELL_INIT_PROMPTED` and `~/.tmux.conf` contains the prefix `m` toggle (deployment proof).



Step 3: Re-run tests 51 + 52 against the freshly-deployed artifacts; capture EVIDENCE into `docs/qa/<run-id>/mistborn/`.



Step 4: Restart the live Herald session politely IF the operator confirms (it currently runs the old config); otherwise note that running servers keep the old config until restarted (do not kill an attached session without authorization — §9 / outward-facing caution).

Task 8: Deploy + validate on nezha (Linux)



Step 1: `ssh milosvasic@10.6.100.146` (via ~/nezha); `cd ~/Projects/tmux`; `git fetch --all --prune && git pull --ff-only` (§11.4.37/§11.4.71) to land all fixes.



Step 2: `bash scripts/install_deps.sh` (Linux needs root for deps) then `bash scripts/setup.sh`.



Step 3 (Bug #2 decisive on the affected host): re-run the PTY double-prompt reproduction on nezha (`bash -l -i`) → assert `PROMPT_COUNT == 1` now (was 2). Run `scripts/tests/51_*.sh`. Capture into `docs/qa/<run-id>/nezha/`.



Step 4: Run `scripts/tests/run_all.sh` + meta-test on nezha; assert GREEN / reasoned-SKIP only.



Step 5 (M22): run `bash scripts/codegraph_setup.sh` on nezha to repair the CodeGraph baseline; re-run meta-test → M22 now CAUGHT (not escaped) or honest SKIP; record evidence.

Task 9: Documentation sync (§11.4.12/§11.4.44/§11.4.59/§11.4.60/§11.4.65/§11.4.83/§11.4.99)



Step 1: Update `docs/guide mouse/clipboard` section + `docs/.../clipboard.md`: document Shift-drag (tracking apps), prefix `m toggle`, native-selection path. §11.4.99: cross-reference against the latest tmux 3.6 manual online (mouse/copy-mode) and add a `## Sources verified 2026-05-29` footer + commit-message `Sources verified` footer.



Step 2: Migrate resolved Issues→Fixed (Bug #1, Bug #2, B3 portions) with ATM-NNN, Type, Status, captured-evidence paths; regenerate summaries + HTML/PDF.



Step 3: Update `CONTINUATION.md` §3, `README.md` doc-link table, Status docs if any.



Step 4: Create `docs/qa/<run-id>/` with the bidirectional transcripts/evidence for each fix (PTY logs, tmux copy proofs, iTerm2 captures, nezha logs).

Task 10: Release



Step 1: Bump VERSION (version= semver + versionCode= monotonic) — minor bump (two user-facing bug fixes + tests).



Step 2: Add CHANGELOG.md entry + comprehensive docs/changelogs/v<new>.md (root causes, fixes, tests, dual-host validation evidence, B3 closure).



Step 3: bash commit_all.sh "v<new> - mouse-copy toggle + double-prompt idempotency guard + B3 closure + dual-host validation".



Step 4: Create GitHub release (gh release create) + GitLab release (glab release create) with the changelog notes. Confirm both published.

Self-Review notes

- Spec coverage: Bug #1 (Tasks 3,4,7,8,9), Bug #2 (Tasks 1,2,7,8), B3 (Task 5, 8.5), per-fix physical-proof tests (Tasks 1,3,4 + suite Tasks 6,8), dual-host setup+validate (Tasks 7,8), docs (Task 9), release (Task 10) — all present.
- Anti-bluff: every fix has a RED→GREEN test with captured EVIDENCE; paired mutation for Bug #2; Bug #1 has binding+pipe proof plus best-effort real-mouse with honest §11.4.3 SKIP.
- No placeholders: test bodies are complete and runnable; adapt only the meta-test helper-call shape to the existing harness conventions (read them first).